Time: 120 minutes Max marks: 30

Instructions:
- Do not plagiarize. Do not assist your classmates in plagiarism.
- Show your full solution for the questions to get full credit.
- Attempt all questions that you can.
- True/False questions require justification for true statements and counter-examples for false ones.
- In the unlikely case a question is not clear, discuss it with an invigilating TA. Please ensure that you clearly include any assumptions you make, even after clarification from the invigilator.

1. (8 points) **State True / False with justification / counter-example [CO-1 to CO4].**
   (a) (1 point) A fully connected neural network with one hidden layer can be used to model the function $f(x) = \sin(x)$.
   (b) (1 point) A 1×1 convolution layer used with unit stride increases the receptive field of the CNN.
   (c) (1 point) The Fundamental Matrix is a rank-2 matrix.
   (d) (1 point) Two sets of parallel lines (each set with a different slope) can intersect at the same point.
   (e) (1 point) The minimal sample subset for estimating the vanishing points using RANSAC is 4.
   (f) (1 point) To estimate the Fundamental Matrix, the reprojection error is minimized.
   (g) (1 point) The non-zero singular values of the Essential matrix are identical.
   (h) (1 point) Positional encodings eliminate permutation equivariance of the Self-Attention mechanism.

Solution:

(a) (1 point) **True**. As per the Universal Approximation Theorem, a single hidden layer with non-linear activations can approximate any continuous function, including $\sin(x)$

(b) (1 point) **False**. A 1×1 convolution with unit stride only processes individual neurons across channels and does not aggregate spatial information from neighboring neurons, therefore it maintains the receptive field of the previous layer.

(c) (1 point) **True**. The Fundamental Matrix is singular because it is constructed by multiplying the full-rank intrinsic parameter matrices (or their inverse) with the Essential Matrix, which in turn is a rank-2 matrix. The Essential matrix has rank-2 due to its construction by multiplying a cross-product matrix (rank-2, owing to its 1-D null space) and a full-rank 3D rotation matrix.

(d) (1 point) **False**. Distinct sets of parallel lines intersect at distinct vanishing points. They only meet at the same point if all lines are parallel to each other.

(e) (1 point) **False**. Since a vanishing point is the intersection of parallel lines, the minimal subset is *2 lines*.

(f) (1 point) **False**. The Fundamental Matrix is typically estimated by minimizing the residual error computed using the epipolar constraint via the 8-point algorithm ($\mathbf{x}^\top \mathbf{F} \mathbf{x}' = 0$), also called as the algebraic error.

(g) (1 point) **True**. For a calibrated camera, the Essential Matrix is $[\mathbf{t}]_\times \mathbf{R}$, and the skew-symmetric matrix $[\mathbf{t}]_\times$ has two equal non-zero singular values.

(h) (1 point) **True**. By incorporating unique position-dependent values to input embeddings, the model can distinguish between different sequences of the same tokens, breaking the inherent symmetry.

Rubric: 0 if incorrect, 0.5 answer, 0.5 explanation

2. (7 points) **Deep Neural Networks for Visual Recognition [CO-1].**
   (a) (2 points) Let $\mathbf{x} = [x_1, x_2, x_3]^\top, \mathbf{z} = [z_1, z_2, z_3]^\top$ and $\mathbf{a} = [a_1, a_2, a_3]^\top$ be the input, logits and the output of a fully connected layer with a ReLU activation, respectively. Let $\mathbf{W}$ denote the weights (assume there are no bias parameters) and $\mathcal{L}$ be the loss function that is computed using the activation $\mathbf{a}$. Write the expression of the gradient $\frac{\partial \mathcal{L}}{\mathbf{W}}$ in terms of $\mathbf{x}, \mathbf{z}$, and $\mathbf{a}$ that would be computed during backpropagation for updating $\mathbf{W}$.
   (b) (2 points) The Feature Pyramid Network in the YOLOv8 model aims to retain both spatial resolution *and* semantic information. How does it accomplish learning semantic information? How does the architecture retain spatial information? Comment on what kind of architectural element enables the merging of spatial and semantic information.
   (c) (3 points) Assume a calibrated and rectified stereo camera setup that captures a pair of images. The objective is to predict the depth map image. Design a CNN-based encoder-decoder model, which uses the encoder to extract features that feeds into a single attention layer for solving the correspondence problem, and a decoder network that outputs the disparity / depth map at the same resolution as the input images. Describe the role of the attention layer and the decoder in this architecture, including what would be the tokens, and the type of convolution layer appropriate for the decoder. {*Hint*: Recall that rectified images have corresponding points in the same row.}

Solution:

(a) (2 points) The forward pass for the fully connected layer is defined as:

$$\mathbf{z} = \mathbf{W}\mathbf{x}$$
$$\mathbf{a} = \text{ReLU}(\mathbf{z}) = \max(0, \mathbf{z})$$

Using the chain rule, the gradient of the loss $\mathcal{L}$ with respect to the weight matrix $\mathbf{W}$ is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{W}}$$

First, we define the element-wise gradient of the ReLU activation, $\sigma'(\mathbf{z})$. For each component z_i:

$$\frac{\partial a_i}{\partial z_i} = \begin{cases} 1 & \text{if } z_i > 0 \\ 0 & \text{if } z_i \leq 0 \end{cases}$$

Applying the chain rule to the logits, we get the vector:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \\ \frac{\partial \mathcal{L}}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \\ \frac{\partial \mathcal{L}}{\partial a_3} \cdot \frac{\partial a_3}{\partial z_3} \end{bmatrix} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}} \odot \begin{bmatrix} \mathbb{1}_{z_1>0} \\ \mathbb{1}_{z_2>0} \\ \mathbb{1}_{z_3>0} \end{bmatrix}$$

Finally, combining this with the derivative of the linear operation $\frac{\partial \mathbf{z}}{\partial \mathbf{W}}$, we obtain the full expression for the gradient:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{a}} \odot \begin{bmatrix} \mathbb{1}_{z_1>0} \\ \mathbb{1}_{z_2>0} \\ \mathbb{1}_{z_3>0} \end{bmatrix} \right) \mathbf{x}^\top$$

Alternatively, expressing each entry (i, j) of the gradient matrix:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}} = \begin{cases} \frac{\partial \mathcal{L}}{\partial a_i} x_j & \text{if } z_i > 0 \\ 0 & \text{if } z_i \leq 0 \end{cases}$$

Rubric:
0.5 partial derivative wrt w
0.5 partial derivative wrt z
1 final expression

(b) (2 points) The FPN architecture retains semantic information and spatial details by

1. **Semantic Information**. Extracting features as the image passes through the backbone with the deeper layers gradually capture high-level semantic features while reducing spatial resolution.
2. **Spatial Resolution**. The spatial resolution is maintained via upsampling the semantically rich feature maps from the deeper layers
3. **Fusion**. Fusing it with the original high-resolution feature maps from the shallower layers, by leveraging lateral connections, 1×1 convolutions and elementwise additions.

Rubric:
Define
0.5 points if deep layers for semantic features are mentioned;
0.5 points if leveraging higher-resolution feature maps from shallower layers is mentioned;
Fusion:
0.5 points if fusion and/or upsampling is mentioned;
0.5 if 1×1 convolutions are mentioned.

(c) (3 points) The following is an example of an architecture that could be used for the depth/disparity map prediction problem.

1. **Encoder**. A feedforward CNN based encoder that has convolutional layers with nonlinear activations (e.g., ReLU) that progressively subsamples the feature maps (via pooling layers or larger strides). The same encoder processes both images from the stereo pair.
2. **Attention Layer**. The attention layer could be used to identify correlations between pixels (patches) for identifying point correspondence scores by using the features extracted from the Encoder as the tokens. Furthermore, given the rectified images, the attention can be reduced to 1-D attention by computing weights only along corresponding rows.
3. **Decoder**. The Decoder would leverage attention maps and feature embeddings to progressively upsample to the original spatial resolution by leveraging transposed or atrous convolutions along with non-linear activation and normalization layers.

Rubric: 1 each for describing the Encoder, Attention (with rectified images to restricting the attention to 1D mentioned) and Decoder (with appropriate upsampling layers mentioned) components.

3. (5 points) **Solving Computer Vision Problems [CO-3].**
You are given many dashcam videos (a camera on a car's dashboard) that has been collected from city roads and highways. Assume that the camera is calibrated, has no roll or yaw, and has a fixed pitch angle w.r.t. the ground plane. You may assume all videos are collected on planar roads, however, only RGB videos are available and no depth information is present. You have access to a near perfectly trained object detector (e.g. YOLOv8) and a multi-object tracker (e.g. ByteTrack) for tracking all objects of interest. Your objective is to use the aforementioned data and models / tools available to train a model for *object trajectory forecasting* in 3D. You are **not** permitted to annotate any videos for training. Describe in as much detail as you can, how would you train and evaluate your trajectory forecasting model? Specifically, describe (1) Any pre-processing of images / frames necessary. (2) Training setup and

methodology. (3) Loss functions. (4) Training / Validation split. (5) Evaluation setup and methodology.

Solution: To train a 3D object trajectory forecasting model without manual annotations using calibrated dashcam footage, you can leverage the *ground plane constraint* and *temporal self-supervision*. Since the road is planar and the camera is calibrated with a fixed pitch, every 2D pixel on the road has a unique 3D coordinate in the world. An example solution is described below:

1. **Pre-processing**. YOLOv8 and ByteTrack could be used to detect and track all objects of interest in all videos. Calibration could be used to compute a homography between the image and the ground plane to generate trajectories in 3D (or after projection to the 2D ground plane). The trajectory prediction could be predicted with respect to the dashcam's location (ego-vehicle's location), i.e., predicting relative motion.
2. **Training Setup and Methodology**. Since manual annotations for training is not permitted, a self-supervised learning framework needs to be used. Given a good detection and tracking model, it is reasonable to assume that the detected trajectories by YOLOv8+ByteTrack have acceptable accuracy.
   - Model input / output: The problem of trajectory forecasting is to take a history of trajectory as input and predict the forecasted trajectory as output (for k timesteps in the future).
   - Architecture: Any sequence-based architecture, e.g., a Transformer or a recurrent neural network can be used.
   - Labels: The labels can be the object trajectories taken from the pre-processed videos and the pretext task be defined as taking historical trajectories and predicting the future trajectories.
3. **Loss Functions**. The loss function could be an MSE, MAE, RMSE loss, possibly with a regularization term that penalizes arbitrary / unrealistic jumps in trajectories.
4. **Training / Validation split**. A 70/30 (or a similar ratio) is typically used. However, it is crucial to split in a manner that segments from the same video (or more strictly, scene) do not appear both in the training and validation sets.
5. **Evaluation setup and methodology**. The evaluation metrics should be based on discrepancy between the predicted and the actual trajectory (as obtained in the pre-processing stage). An L_2 norm of the difference in the predicted an actual trajectory could serve as an appropriate metric. More popularly used metrics are Average Displacement Error (ADE), which is the average Euclidean distance between the points of predicted and actual trajectories, usually paired with Final Displacement Error (FDE), which is the Euclidean distance between the final locations of the predicted and actual trajectories.

Rubric: 1 point for each of the five components.
1. Pre-processing
2. Model
3. Loss Function (accepted answers)
4. The train/val split should carry a penalty if the video / scene segregation is not considered.
5. Setup

Other components can carry partial grading based on the answers.

4. (5 points) **Geometry [CO-2]**

(a) (1 point) What is the difference between Structure from Motion (SfM) and Simultaneous Localization and Mapping (SLAM)?

(b) (2 points) What is drift in the context of SLAM? How is it mitigated? Does SfM also suffer from this problem? If yes, how do SfM approaches handle it?

(c) (2 points) Derive the Rodriguez's formula for computing the Rotation Matrix, given the axis and angle of rotation.

Solution:

(a) (1 point) SfM (Structure from Motion): This is typically an offline/batch process. All images are collected first (e.g., a thousand photos of a building) and then process them all at once to create a highly accurate 3D model.
SLAM (Simultaneous Localization and Mapping): This is an online/real-time process. The system builds the map and determines its position simultaneously as the sensor moves (e.g., a mobile robot or a drone moving through a room).

(b) (2 points) In the context of SLAM, drift is the gradual accumulation of small errors in the estimation of a robot's pose (position and orientation) over time. Because SLAM usually estimates the current state based on the previous state, tiny inaccuracies in frame-to-frame tracking compound, leading to a significant divergence between the estimated trajectory and the actual path.

Drift in SLAM is handled via Loop Closure and Local Bundle Adjustment. The former identifies a loop and performs a global bundle adjustment (optimization over the entire loop) to adjust the intermediate poses to minimize drift. Local Bundle Adjustment optimizes poses and 3D points over a local *window* of keyframes.

SfM also suffers from drift, especially when videos are used for reconstruction. This is usually handled by Global Bundle Adjustment that optimizes over camera poses, and reconstructed points using nonlinear optimization techniques that minimize an objective like the Reprojection Error. When external information is available, e.g., ground control points (like GPS coordinates), those can be used to further correct the errors due to drift.

Rubric:
1 for drift in context of SLAM
0.5 for any one drift mitigation technique
0.25 for drift in SfM if Present
0.25 approach to handle SfM if Present

(c) (2 points) Refer to Slide Set 05 (slides 54-56) for the derivation.

5. (5 points) **Evaluation Metrics [CO-4]**

(a) (1 point) Write the expression for the Reprojection error.

(b) (2 points) Give three examples of computer vision applications where it makes sense to minimize the reprojection error.

(c) (2 points) How would you evaluate the effectiveness of RANSAC? Given 30% inliers and 70% outliers, how many iterations of RANSAC would you need to perform in order to successfully estimate a (1) Planar Homography and (2) Fundamental Matrix.

Solution:

(a) (1 point) The **Reprojection Error** is a measure of the geometric distance between an observed image point and the projection of a 3D world point onto the image plane. It is the standard objective function minimized in many geometric computer vision applications.

For a set of n camera poses and m 3D points, the total reprojection error E_R is expressed as:

$$E_R(\{P_i\}, \{\mathbf{X}_j\}) = \sum_{i=1}^{n} \sum_{j=1}^{m} \mathbb{1}_{ij} \left\| \mathbf{x}_{ij} - \pi(P_i, \mathbf{X}_j) \right\|^2 \tag{1}$$

Where:

- $\mathbf{x}_{ij}$ is the observed 2D pixel coordinate of point j in image i.
- $\mathbf{X}_j$ is the estimated 3D coordinate of point j in the world frame.
- P_i represents the camera parameters for frame i (in general, including intrinsics K and extrinsics R, t).
- $\pi(P_i, \mathbf{X}_j)$ is the projection function that maps the 3D point $\mathbf{X}_j$ to the 2D image plane using parameters P_i, typically defined as:

  $$\hat{\mathbf{x}}_{ij} = K[R_i|t_i]\mathbf{X}_j$$

  (followed by a conversion from homogeneous to Cartesian coordinates).
- $\mathbb{1}_{ij}$ is a binary indicator variable where $\mathbb{1}_{ij} = 1$ if point j is visible in image i, and 0 otherwise.
- $\| \cdot \|^2$ denotes the squared L_2 norm (Euclidean distance).

Rubric: 0.5 for Expression for Reprojection, 0.5 For defining terms

(b) (2 points) Applications where it makes sense to minimize the reprojection error.

1. Structure from Motion or 3D reconstruction. Here the 3D points are estimated along with all camera parameters, given only a set of images.
2. Simultaneous Localization and Mapping (SLAM): A sequence of images is given and the objective is to estimate the sequence of camera location and orientation along with maintaining a map of 3D keypoints that can be used again for localization.
3. Camera Calibration: Here 3D points are given and the camera parameters are estimated by minimizing the reprojection error, i.e., the distance between the projection of the 3D points on the images and the corresponding image points that are detected in the image.

Rubric: 0.75 SLAM, 0.75 Calibration, 0.5 Any else

(c) (2 points) The probability of success p represents the likelihood that at least one random sample of size s contains only inliers. To achieve a desired confidence p, the required number of iterations N is given by:

$$N = \frac{\log(1-p)}{\log(1-w^s)} \tag{2}$$

Parameters:

- p: Desired probability of success (commonly set to 0.99).
- w: Inlier ratio (0.30 in this case).
- s: Minimum number of points required to estimate the model.

1. Planar Homography.
Estimating a homography requires a minimum of $s = 4$ point correspondences.

- $w^4 = 0.3^4 = 0.0081$
- $N = \frac{\log(1-0.99)}{\log(1-0.0081)} \approx \frac{-2}{-0.00353} \approx \mathbf{566}$

2. Fundamental Matrix.
Using the standard 8-point algorithm, the sample size is $s = 8$.

- $w^s = 0.3^8 \approx 0.00006561$
- $N = \frac{\log(1-0.99)}{\log(1-0.00006561)} \approx \frac{-2}{-0.0000285} \approx \mathbf{70,175}$

Evaluation: RANSAC is evaluated based on the **final inlier count** (maximizing support), **stability** of the result across runs, and the **RMSE** of the model relative to the identified inlier set.

Rubric: 0.5 Planer, 0.5 Fundamental, 1.0 Evaluate effectiveness of RANSAC,